

M2 MIC - Codes correcteurs - TD/TP 5

Exercice 1 : Construction d'un code LDPC régulier, la méthode de Gallager

Objectif : étant donnés n, d_v et d_c , obtenir une matrice $H \in \mathcal{M}_{m,n}(\mathbb{F}_2)$ contenant exactement d_c « 1 » par ligne et d_v « 1 » par colonne.

- On sépare les m lignes de H en d_v ensembles de $\frac{m}{d_v}$ lignes.
 - Le premier ensemble de lignes est constitué de d_c « 1 » consécutifs répartis sur les $\frac{m}{d_v}$ lignes.
 - Tous les autres ensembles de lignes sont constitués d'une permutation aléatoire des colonnes du premier ensemble de lignes.
1. Quelles contraintes doit-on imposer à m et n pour que la procédure soit réalisable ?
 2. Lorsque les contraintes sont réalisées, vérifier que la procédure proposée permet d'atteindre l'objectif.
 3. Implémenter la fonction `LDPC_gallager(n, d_v, d_c)` sur Sage Math.

Exercice 2 : Construction d'un code LDPC régulier, la méthode de MacKay et Neal

L'objectif est le même que l'exercice précédent.

- On initialise la matrice H avec une première colonne contenant d_v « 1 » pris de façon aléatoire.
- On rajoute des colonnes au fur et à mesure en choisissant l'emplacement des « 1 » de façon aléatoire dans les lignes qui n'ont pas encore d_c « 1 »

Implémenter la fonction `LDPC_MacKay_Neal(n, d_v, d_c)` sur Sage Math.

Exercice 3 : Graphe de Tanner

À toute matrice de parité $H \in \mathcal{M}_{m,n}(\mathbb{F}_2)$ on associe un graphe biparti $G = (V \cup C, E)$, appelé *graphe de Tanner*, où $V = \{v_1, \dots, v_n\}$ désigne les *nœuds variables*, $C = \{c_1, \dots, c_m\}$ les *nœuds de contrôle*, et l'arête $\{v_j, c_i\}$ appartient à E si et seulement si $H_{i,j} = 1$.

Rappels sur l'objet Graph de Sage Math. Sage Math propose une classe `Graph` permettant de manipuler des graphes non orientés. Voici les principales fonctions utiles pour cet exercice.

- **Création d'un graphe vide et ajout de sommets et d'arêtes :**

```
G = Graph()
G.add_vertex("v1")           # ajoute un sommet nommé "v1"
G.add_vertices(["v2","v3"])  # ajoute plusieurs sommets d'un coup
G.add_edge("v1", "v2")       # ajoute l'arête {v1, v2}
G.add_edges([("v1","v3"), ("v2","v3")]) # ajoute plusieurs arêtes
```
- **Création directe depuis une liste d'arêtes :**

```
G = Graph([("v1","v2"), ("v1","v3"), ("v2","c1")])
```
- **Informations de base :**

```
G.vertices()  # liste des sommets
G.edges()     # liste des arêtes sous la forme (u, v, label)
G.num_verts() # nombre de sommets
G.num_edges() # nombre d'arêtes
G.degree("v1") # degré du sommet "v1"
```
- **Propriétés structurelles :**

```

G.is_bipartite()      # True/False
G.bipartite_sets()    # renvoie les deux parties (U, V) du graphe
G.girth()             # longueur du plus petit cycle (girth)
G.is_connected()      # True/False

```

— Visualisation :

La méthode `.plot()` renvoie un objet graphique que l'on peut afficher directement dans Sage. On aussi colorer les sommets pour distinguer les nœuds variables des nœuds de contrôle :

```

color_dict = {"red":  ["v"+str(j) for j in range(n)],
              "blue":  ["c"+str(i) for i in range(m)]}
G.plot(vertex_colors=color_dict, layout='bipartite')

```

On peut forcer manuellement la position des sommets avec l'option `pos`, qui associe à chaque sommet un couple de coordonnées (x, y) . Cela permet par exemple de placer les nœuds variables sur une colonne à gauche et les nœuds de contrôle sur une colonne à droite :

```

pos = {}
for j, v in enumerate(var_nodes):
    pos[v] = (0, -j)
for i, c in enumerate(ctrl_nodes):
    pos[c] = (4, -i * n / m)
G.plot(vertex_colors=color_dict, pos=pos)

```

Le facteur n/m sur les nœuds de contrôle permet d'aligner les deux colonnes sur la même plage verticale lorsque $m \neq n$.

Questions.

1. Construire une fonction `tanner_graph(H)` qui, à partir d'une matrice $H \in \mathcal{M}_{m,n}(\mathbb{F}_2)$, renvoie le graphe de Tanner associé sous la forme d'un objet `Graph` de Sage. On nommera les nœuds variables "v0", ..., "v{n-1}" et les nœuds de contrôle "c0", ..., "c{m-1}".
Vérifier que le graphe obtenu est bien biparti avec `.is_bipartite()`, et que les degrés des nœuds variables et des nœuds de contrôle valent respectivement d_v et d_c .
2. Pour $n = 12$, $d_v = 3$, $d_c = 4$, afficher un graphe de Tanner en coloriant les nœuds variables en rouge et les nœuds de contrôle en bleu à l'aide de l'option `vertex_colors`.

Exercice 4 : Décodage par *bit-flipping*

On considère un canal binaire symétrique sans mémoire (BSC) de probabilité d'erreur p (chaque lettre de l'erreur est choisie selon une loi de Bernoulli de paramètre p). Soit H la matrice de parité d'un code LDPC de paramètres (n, d_v, d_c) , $x \in \mathbb{F}_2^n$ un mot du code et $y = x + e$ le mot reçu, où e est le vecteur d'erreur.

L'algorithme de *bit-flipping* a été décrit en cours.

1. Implémenter la fonction `bit_flipping(H, y, max_iter)` sur Sage Math.
2. Pour $d_v = 3$, $d_c = 6$ et $n \in \{120, 600, 1200\}$, générer une matrice H par la méthode de votre choix, choisir un mot du code aléatoire x , le corrompre par un BSC de paramètre p , puis décoder avec l'algorithme de `bit_flipping`. Estimer l'efficacité du décodage (réussite, échec) en fonction de p . On pourra parallèlement estimer le temps de calcul lors de la phase de décodage.